

Practical Week – 9

Task 1 - Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <sys/time.h>

#define BUFFER_SIZE 4096

/* -----
PART 1: File Copy using Linux System Calls
open(), read(), write(), lseek(), close()
----- */

void system_call_copy(const char *source, const char *destination) {
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytesRead, bytesWritten;

    src = open(source, O_RDONLY);

    if (src == -1) {
        perror("Error opening source file");
        return;
    }

    dest = open(destination,
                 O_WRONLY | O_CREAT | O_TRUNC,
                 0644);

    if (dest == -1) {
        perror("Error opening destination file");
        close(src);
        return;
    }

    /* Using lseek() to find the file size */
    off_t fileSize = lseek(src, 0, SEEK_END);

    if (fileSize == -1) {
        perror("lseek error");
        close(src);
        close(dest);
        return;
    }

    printf("\nFile size using lseek(): %ld bytes\n",
           (long)fileSize);

    /* Reset file position to beginning */
```

```

/* Reset file position to beginning */
lseek(src, 0, SEEK_SET);

/* Copy file */
while ((bytesRead = read(src, buffer, BUFFER_SIZE)) > 0) {
    bytesWritten = write(dest, buffer, bytesRead);

    if (bytesWritten != bytesRead) {
        perror("write error");
        break;
    }
}

if (bytesRead == -1) {
    perror("read error");
}

close(src);
close(dest);

printf("File copied using Linux system calls.\n");
}

/* -----
PART 2: File Copy using Standard Library
fopen(), fread(), fwrite(), fclose()
----- */

void standard_library_copy(const char *source,
                           const char *destination) {

    FILE *src;
    FILE *dest;

    char buffer[BUFFER_SIZE];

    size_t bytesRead;
    size_t bytesWritten;

    src = fopen(source, "rb");

    if (src == NULL) {
        perror("Error opening source file");
        return;
    }

    dest = fopen(destination, "wb");

    if (dest == NULL) {

```

```

    if (dest == NULL) {
        perror("Error opening destination file");
        fclose(src);
        return;
    }

    while ((bytesRead = fread(buffer, 1, BUFFER_SIZE, src)) > 0) {
        bytesWritten = fwrite(buffer, 1, bytesRead, dest);

        if (bytesWritten != bytesRead) {
            perror("fwrite error");
            break;
        }
    }

    fclose(src);
    fclose(dest);

    printf("File copied using standard library functions.\n");
}

/* -----
PART 3: Performance Comparison
----- */

double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

void performance_test(const char *source) {
    double start, end;

    printf("\n===== PERFORMANCE TEST =====\n");

    /* System call copy */
    start = get_time();

    system_call_copy(source, "system_copy.bin");

    end = get_time();
}

```

```

    end = get_time();

    double systemTime = end - start;

    printf("System call copy time : %.6f seconds\n",
           systemTime);

    /* Standard library copy */
    start = get_time();

    standard_library_copy(source, "library_copy.bin");

    end = get_time();

    double libraryTime = end - start;

    printf("Standard library copy time : %.6f seconds\n",
           libraryTime);

    printf("=====\n");
}

/* -----
PART 4: I/O Redirection using dup2()
----- */

void io_redirection() {
    int input_fd;
    int output_fd;

    char buffer[BUFFER_SIZE];
    ssize_t bytesRead;

    input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1) {
        perror("Error opening input.txt");
        return;
    }

    output_fd = open("output.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

    if (output_fd == -1) {
        perror("Error opening output.txt");
        close(input_fd);
    }
}

```

```

        close(input_fd);
        return;
    }

    /*
    Redirect standard input (0)
    from input.txt
    */
    if (dup2(input_fd, STDIN_FILENO) == -1) {
        perror("dup2 input error");
        return;
    }

    /*
    Redirect standard output (1)
    to output.txt
    */
    if (dup2(output_fd, STDOUT_FILENO) == -1) {
        perror("dup2 output error");
        return;
    }

    close(input_fd);
    close(output_fd);

    /*
    Read from STDIN and write to STDOUT.
    Because of dup2(), these now refer
    to input.txt and output.txt.
    */

    while ((bytesRead = read(STDIN_FILENO,
                            buffer,
                            BUFFER_SIZE)) > 0) {

        write(STDOUT_FILENO, buffer, bytesRead);
    }
}

/* -----
MAIN FUNCTION
----- */

int main() {

    int choice;

    char source[100];
    char destination[100];

```

```
printf("=====\n");
printf("      LINUX FILE I/O PROJECT\n");
printf("=====\n");

printf("\n1. File Copy using System Calls\n");
printf("2. File Copy using Standard Library\n");
printf("3. Performance Comparison\n");
printf("4. I/O Redirection using dup2()\n");
printf("5. Exit\n");

printf("\nEnter your choice: ");
scanf("%d", &choice);

switch (choice) {

    case 1:

        printf("Enter source file: ");
        scanf("%99s", source);

        printf("Enter destination file: ");
        scanf("%99s", destination);

        system_call_copy(source, destination);

        break;

    case 2:

        printf("Enter source file: ");
        scanf("%99s", source);

        printf("Enter destination file: ");
        scanf("%99s", destination);

        standard_library_copy(source, destination);

        break;

    case 3:

        printf("Enter source file: ");
        scanf("%99s", source);

        performance_test(source);

        break;
```

```

        case 4:
            /*
             * Create input.txt before selecting this option.
             */
            io_redirection();
            break;

        case 5:
            printf("Program exited.\n");
            break;

        default:
            printf("Invalid choice.\n");
    }

    return 0;
}

```

```

mohana_priya@AMMU: ~/tas  x  +  v
mohana_priya@AMMU:~$ mkdir task30
cd task30
mohana_priya@AMMU:~/task30$ nano task30.c
mohana_priya@AMMU:~/task30$
mohana_priya@AMMU:~/task30$ gcc task30.c -o task30
mohana_priya@AMMU:~/task30$ ls
task30  task30.c
mohana_priya@AMMU:~/task30$ echo "This is my Linux File I/O project." > source.txt
mohana_priya@AMMU:~/task30$ cat source.txt
This is my Linux File I/O project.
mohana_priya@AMMU:~/task30$ ./task30
=====
      LINUX FILE I/O PROJECT
=====

1. File Copy using System Calls
2. File Copy using Standard Library
3. Performance Comparison
4. I/O Redirection using dup2()
5. Exit

Enter your choice: 1
Enter source file: source.txt
Enter destination file: copy.txt

File size using lseek(): 35 bytes
File copied using Linux system calls.
mohana_priya@AMMU:~/task30$ cat copy.txt
This is my Linux File I/O project.
mohana_priya@AMMU:~/task30$ ./task30
=====
      LINUX FILE I/O PROJECT
=====

1. File Copy using System Calls
2. File Copy using Standard Library
3. Performance Comparison
4. I/O Redirection using dup2()
5. Exit

```

```

Enter your choice: 2
Enter source file: source.txt
Enter destination file: copy2.txt
File copied using standard library functions.
mohana_priya@AMMU:~/task30$ dd if=/dev/zero of=largefile.bin bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.179401 s, 586 MB/s
mohana_priya@AMMU:~/task30$ ./task30
=====
LINUX FILE I/O PROJECT
=====

1. File Copy using System Calls
2. File Copy using Standard Library
3. Performance Comparison
4. I/O Redirection using dup2()
5. Exit

Enter your choice: 3
Enter source file: largefile.bin

===== PERFORMANCE TEST =====

File size using lseek(): 104857600 bytes
File copied using Linux system calls.
System call copy time : 0.228861 seconds
File copied using standard library functions.
Standard library copy time : 0.240258 seconds
=====
mohana_priya@AMMU:~/task30$ echo "Hello from input.txt. This demonstrates du
p2 I/O redirection." > input.txt
mohana_priya@AMMU:~/task30$ ./task30
=====
LINUX FILE I/O PROJECT
=====

1. File Copy using System Calls
2. File Copy using Standard Library
3. Performance Comparison

```

```

=====
LINUX FILE I/O PROJECT
=====

1. File Copy using System Calls
2. File Copy using Standard Library
3. Performance Comparison
4. I/O Redirection using dup2()
5. Exit

Enter your choice: 4
mohana_priya@AMMU:~/task30$ cat output.txt
Hello from input.txt. This demonstrates dup2 I/O redirection.
mohana_priya@AMMU:~/task30$ cat < input.txt > output2.txt
mohana_priya@AMMU:~/task30$ cat output2.txt
Hello from input.txt. This demonstrates dup2 I/O redirection.
mohana_priya@AMMU:~/task30$ |

```

Task 2 - Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int input_fd;
    int output_fd;
    char buffer[100];
    ssize_t bytes_read;

    /* Open input file */
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1) {
        perror("Error opening input.txt");
        return 1;
    }

    /* Open output file */
    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd == -1) {
        perror("Error opening output.txt");
        close(input_fd);
        return 1;
    }

    /*
     * Redirect standard input (file descriptor 0)
     * to input.txt
     */
    if (dup2(input_fd, STDIN_FILENO) == -1) {
        perror("dup2 input error");
        return 1;
    }

    /*
     * Redirect standard output (file descriptor 1)
     * to output.txt
     */
    if (dup2(output_fd, STDOUT_FILENO) == -1) {
        perror("dup2 output error");
        return 1;
    }

    /* Original file descriptors are no longer needed */
    close(input_fd);
    close(output_fd);

    /*
     * Read from standard input.
     * Because of dup2(), STDIN now means input.txt.
     */
    while ((bytes_read = read(STDIN_FILENO,
                             buffer,
                             sizeof(buffer))) > 0) {

        /*
         * Write to standard output.
         * Because of dup2(), STDOUT now means output.txt.
         */
        write(STDOUT_FILENO, buffer, bytes_read);
    }

    return 0;
}
```

```
mohana_priya@AMMU:~/task30$ cat output.txt
Hello from input.txt. This demonstrates dup2 I/O redirection.
mohana_priya@AMMU:~/task30$ cat < input.txt > output2.txt
mohana_priya@AMMU:~/task30$ cat output2.txt
Hello from input.txt. This demonstrates dup2 I/O redirection.
mohana_priya@AMMU:~/task30$ mkdir task31
cd task31
mohana_priya@AMMU:~/task30/task31$ nano task31.c
mohana_priya@AMMU:~/task30/task31$ gcc task31.c -o task31
mohana_priya@AMMU:~/task30/task31$ ls
task31  task31.c
mohana_priya@AMMU:~/task30/task31$ echo "Hello! This is Task 31 demonstrating I/O redirection using dup2()." > input.txt
mohana_priya@AMMU:~/task30/task31$ cat input.txt
Hello! This is Task 31 demonstrating I/O redirection using dup2().
mohana_priya@AMMU:~/task30/task31$ ./task31
mohana_priya@AMMU:~/task30/task31$ cat output.txt
Hello! This is Task 31 demonstrating I/O redirection using dup2().
mohana_priya@AMMU:~/task30/task31$ cat < input.txt
Hello! This is Task 31 demonstrating I/O redirection using dup2().
mohana_priya@AMMU:~/task30/task31$ cat < input.txt > output2.txt
mohana_priya@AMMU:~/task30/task31$ cat output2.txt
Hello! This is Task 31 demonstrating I/O redirection using dup2().
```